Smart farming system

```
package Smartfarming_buffer;
import java.util.*;
//==========================================================
//UNIFIED CROP CLASS — merges all properties from all modules
//==========================================================
class Crop {
// Basic info
String name;
String soilType;
String season;
String climate;
// Temperature range
double minTemp, maxTemp, idealTemp;
// Water & fertility
double waterNeed;       // 0–100 (relative)
double minRainfall;     // mm
double minMoisture;     // %
double minHumidity;     // %
double fertilityNeed;   // 0–1
// Yield & growth
double avgYield;        // tonnes/hectare
int    growthDays;
String npkNeeds;
// Market data
double marketDemand;    // 0–1 (used in scoring)
int    demandScore;     // 0–100
int    marketPrice;     // ₹/quintal
int    priceTrend;      // 1=Rising, 0=Stable, -1=Falling
boolean exportDemand;
// Computed score (used in recommendation ranking)
double score;
Crop(String name, String soilType, String season, String climate,
     double minTemp, double maxTemp, double idealTemp,
     double waterNeed, double minRainfall, double minMoisture, double minHumidity,
     double fertilityNeed, double avgYield, int growthDays, String npkNeeds,
     double marketDemand, int demandScore, int marketPrice, int priceTrend, boolean
exportDemand) {
    this.name        = name;
    this.soilType    = soilType;
    this.season      = season;
    this.climate     = climate;
    this.minTemp     = minTemp;
    this.maxTemp     = maxTemp;
    this.idealTemp   = idealTemp;
    this.waterNeed   = waterNeed;
    this.minRainfall = minRainfall;
    this.minMoisture = minMoisture;
    this.minHumidity = minHumidity;
    this.fertilityNeed = fertilityNeed;
    this.avgYield    = avgYield;
    this.growthDays  = growthDays;
    this.npkNeeds    = npkNeeds;
    this.marketDemand = marketDemand;
    this.demandScore  = demandScore;
    this.marketPrice  = marketPrice;
    this.priceTrend   = priceTrend;
```

```
    this.exportDemand = exportDemand;
    this.score        = 0;
}
}
//==============================================================
//MODULE 1 — ClimateInputHandler
//Reads and validates all user inputs
//==============================================================
class ClimateInputHandler {
static Scanner sc = new Scanner(System.in);
/** Read a non-negative double; keep asking on bad input */
public static double getDoubleInput(String message) {
    while (true) {
        try {
            System.out.print(message);
            double value = sc.nextDouble();
            if (value < 0) throw new InputMismatchException();
            return value;
        } catch (InputMismatchException e) {
            System.out.println("  ✗ Invalid input! Please enter a valid positive number.");
            sc.next();
        }
    }
}
/** Read a trimmed string */
public static String getStringInput(String message) {
    System.out.print(message);
    return sc.next().trim();
}
/** Validate temperature and humidity ranges */
public static boolean validateInputs(double temp, double humidity) {
    if (temp < -10 || temp > 60) {
        System.out.println("  ✗ Invalid Temperature! Must be between -10 and 60 °C.");
        return false;
    }
    if (humidity < 0 || humidity > 100) {
        System.out.println("  ✗ Invalid Humidity! Must be between 0 and 100 %.");
        return false;
    }
    return true;
}
}
//==============================================================
//MODULE 2 — CropRecommendationEngine
//Scores every crop against user conditions, returns ranking
//==============================================================
class CropRecommendationEngine {
/**
 * Calculates a composite score (0–1) for each crop.
 * Weights: soil 30%, temperature 20%, water 20%, market 15%, fertility 15%
 */
public static void calculateScores(HashMap<String, Crop> cropMap,
                                   String soil, double temp,
                                   double water, double fertility) {
    for (Crop c : cropMap.values()) {
        double soilScore      = c.soilType.equalsIgnoreCase(soil) ? 1.0 : 0.5;
        double tempScore      = 1 - (Math.abs(temp - c.idealTemp) / 50.0);
        double waterScore     = 1 - (Math.abs(water - c.waterNeed) / 100.0);
        double fertilityScore = 1 - Math.abs(fertility - c.fertilityNeed);
        c.score = 0.30 * soilScore
```

```
            + 0.20 * tempScore
            + 0.20 * waterScore
            + 0.15 * c.marketDemand
            + 0.15 * fertilityScore;
        // Clamp to [0, 1]
        c.score = Math.max(0, Math.min(c.score, 1));
    }
}
/** Returns crops sorted by score descending (uses ArrayList + Collections.sort) */
public static List<Crop> getTopCrops(HashMap<String, Crop> cropMap) {
    List<Crop> list = new ArrayList<>(cropMap.values());
    Collections.sort(list, (a, b) -> Double.compare(b.score, a.score));
    return list;
}
/** Prints the ranked recommendation list */
public static void displayRecommendations(List<Crop> result) {
    System.out.println("\n╔═══════════════════════════════════════════════════════╗");
    System.out.println("│      🌾 TOP CROP RECOMMENDATIONS        │");
    System.out.println("╚═══════════════════════════════════════════════════════╝");
    System.out.printf("  %-5s %-14s %-8s %-10s%n", "Rank", "Crop", "Score", "Season");
    System.out.println("  ─────────────────────────────────────────────────────");
    for (int i = 0; i < result.size(); i++) {
        Crop c = result.get(i);
        System.out.printf("  %-5d %-14s %-8s %-10s%n",
            (i + 1), c.name,
            String.format("%.2f", c.score),
            c.season);
    }
    System.out.println("  ─────────────────────────────────────────────────────");
}
}
//==========================================================
//MODULE 3 — WaterOptimizationModule
//Gives irrigation advice and drought warnings
//==========================================================
class WaterOptimizationModule {
/** Prints irrigation advice based on available water (mm) */
public static void irrigationAdvice(double waterAvailable) {
    System.out.println("\n╔═══════════════════════════════════════════════════════╗");
    System.out.println("│        💧 IRRIGATION ADVICE             │");
    System.out.println("╚═══════════════════════════════════════════════════════╝");
    if (waterAvailable < 50) {
        System.out.println("  Low water available → Use drip irrigation");
        System.out.println("  & grow drought-resistant crops (Millet, Barley).");
    } else if (waterAvailable < 100) {
        System.out.println("  Moderate water → Irrigate 2–3 times per week.");
    } else {
        System.out.println("  High water → Suitable for water-intensive crops");
        System.out.println("  like Rice and Sugarcane.");
    }
    System.out.println("  ─────────────────────────────────────────────────────");
}
/** Drought warning if both rainfall and moisture are low */
public static void droughtWarning(double rainfall, double moisture) {
    if (rainfall < 40 && moisture < 30) {
        System.out.println("\n  ⚠ DROUGHT CONDITION DETECTED!");
        System.out.println("  Consider drought-resistant varieties and water harvesting.");
    }
}
/** Suggests water-efficient crops when no strong match is found */
```

```java
public static void suggestEfficientCrops(HashMap<String, Crop> cropMap) {
    System.out.println("\n 💧 Suggesting water-efficient crops:");
    for (Crop c : cropMap.values()) {
        if (c.minRainfall < 50) {
            System.out.println("   • " + c.name
                + " | Min Rainfall: " + c.minRainfall + " mm"
                + " | Soil: " + c.soilType);
        }
    }
}
}
//============================================================
//MODULE 4 — PestRiskModule
//Identifies pest/disease risk for the top recommended crop
//============================================================
class PestRiskModule {
/**
 * Checks known pest/disease conditions for a crop.
 * Extend this method to add more crop-disease rules.
 */
public static void checkPestRisk(String cropName, double temp, double humidity) {
    String risk    = "Low";
    String disease = "None";
    if (cropName.equalsIgnoreCase("Rice") && humidity > 75 && temp >= 20 && temp <= 30) {
        risk    = "HIGH";
        disease = "Blast Disease";
    } else if (cropName.equalsIgnoreCase("Wheat") && humidity > 60) {
        risk    = "MEDIUM";
        disease = "Rust Disease";
    } else if (cropName.equalsIgnoreCase("Sugarcane") && humidity > 70) {
        risk    = "MEDIUM";
        disease = "Red Rot";
    } else if (temp > 35 && humidity < 40) {
        risk    = "MEDIUM";
        disease = "Insect Attack (general)";
    }
    System.out.println("\n╔═══════════════════════════════════════════╗");
    System.out.println("║        ⚠ PEST RISK ALERT              ");
    System.out.println("╚═══════════════════════════════════════════╝");
    System.out.println("  Crop Checked   : " + cropName);
    System.out.println("  Pest Risk      : " + risk);
    System.out.println("  Possible Issue : " + disease);
    // Advice based on risk level
    if (risk.equals("HIGH")) {
        System.out.println("  💡 Action: Apply fungicide/pesticide immediately.");
    } else if (risk.equals("MEDIUM")) {
        System.out.println("  💡 Action: Monitor closely; consider preventive spray.");
    } else {
        System.out.println("  ✅ No major pest risk at the moment.");
    }
    System.out.println("  ──────────────────────────────────────────");
}
}
//============================================================
//MODULE 5 — FertilityRecommendationModule
//Advises on fertilizer based on soil fertility level
//============================================================
class FertilityRecommendationModule {
/**
```

```
 * Prints a fertility alert and fertilizer suggestion.
 * @param fertility  value between 0.0 (very poor) and 1.0 (excellent)
 */
public static void fertilityAlert(double fertility) {
    System.out.println("\n╔═══════════════════════════════════════════════════════╗");
    System.out.println("║       🌿 FERTILITY RECOMMENDATION        ║");
    System.out.println("╚═══════════════════════════════════════════════════════╝");
    if (fertility < 0.3) {
        System.out.println("  Fertility Level : LOW  (" + fertility + ")");
        System.out.println("  Suggestion      : Add Nitrogen-rich fertilizer (Urea).");
        System.out.println("  Also consider   : Green manure / compost to rebuild soil.");
    } else if (fertility < 0.6) {
        System.out.println("  Fertility Level : MEDIUM  (" + fertility + ")");
        System.out.println("  Suggestion      : Use balanced NPK fertilizer.");
    } else {
        System.out.println("  Fertility Level : GOOD  (" + fertility + ")");
        System.out.println("  ✅ Soil is healthy. Maintain with organic compost.");
    }
    System.out.println("  ─────────────────────────────────────────────────────");
}
}
//==========================================================
//MODULE 6 — CropInformationModule
//Search by name, filter by season/soil, display crop profile
//==========================================================
class CropInformationModule {
/** Linear Search — finds a crop by name (case-insensitive) */
public static Crop searchByName(HashMap<String, Crop> cropMap, String query) {
    query = query.toLowerCase().trim();
    for (Crop c : cropMap.values()) {
        if (c.name.toLowerCase().equals(query)) {
            return c;
        }
    }
    return null; // not found
}
/** Filter crops whose season matches (or is "All Season") */
public static List<Crop> filterBySeason(HashMap<String, Crop> cropMap, String season) {
    List<Crop> result = new ArrayList<>();
    season = season.toLowerCase().trim();
    for (Crop c : cropMap.values()) {
        if (c.season.toLowerCase().contains(season)
            || c.season.equalsIgnoreCase("All Season")) {
            result.add(c);
        }
    }
    return result;
}
/** Filter crops whose soil type matches */
public static List<Crop> filterBySoil(HashMap<String, Crop> cropMap, String soil) {
    List<Crop> result = new ArrayList<>();
    soil = soil.toLowerCase().trim();
    for (Crop c : cropMap.values()) {
        if (c.soilType.toLowerCase().contains(soil)) {
            result.add(c);
        }
    }
    return result;
}
/** Display detailed profile for one crop */
```

```java
public static void displayCropProfile(Crop c) {
    System.out.println("\n╔═══════════════════════════════════════════════════════════╗");
    System.out.println("│        🌾 CROP INFORMATION PROFILE        │");
    System.out.println("└═══════════════════════════════════════════════════════════╝");
    System.out.println("  Crop Name       : " + c.name);
    System.out.println("  Season          : " + c.season);
    System.out.println("  Soil Type       : " + c.soilType);
    System.out.println("  Climate Needed  : " + c.climate);
    System.out.println("  Temp Range      : " + c.minTemp + "–" + c.maxTemp + " °C");
    System.out.println("  Water Required  : " + c.minRainfall + " mm (min rainfall)");
    System.out.println("  Growth Duration : " + c.growthDays + " days");
    System.out.println("  NPK Fertilizer  : " + c.npkNeeds + " (kg/hectare)");
    System.out.println("  Average Yield   : " + c.avgYield + " tonnes/hectare");
    System.out.println("  ─────────────────────────────────────────────────────────");
}
/** Display a compact table of a list of crops */
public static void displayCropList(List<Crop> crops, String filterLabel) {
    if (crops.isEmpty()) {
        System.out.println("\n  ⚠ No crops found for: " + filterLabel);
        return;
    }
    System.out.println("\n ┌─ Crops matching [" + filterLabel + "] ──────────────────────┐ ");
    System.out.printf("  %-14s %-10s %-10s %-10s%n",
        "Crop", "Season", "Soil", "Water(mm)");
    System.out.println("  ─────────────────────────────────────────────────────────");
    for (Crop c : crops) {
        System.out.printf("  %-14s %-10s %-10s %-10.0f%n",
            c.name, c.season, c.soilType, c.minRainfall);
    }
    System.out.println(" └──────────────────────────────────────────────────────────┘ ");
}
/** Display all crops in a summary table */
public static void displayAllCrops(HashMap<String, Crop> cropMap) {
    System.out.println("\n ┌─ ALL CROPS IN DATABASE ──────────────────────────────┐ ");
    System.out.printf("  %-3s %-14s %-10s %-10s %-8s%n",
        "No", "Crop", "Season", "Soil", "Growth");
    System.out.println("  ─────────────────────────────────────────────────────────");
    int no = 1;
    for (Crop c : cropMap.values()) {
        System.out.printf("  %-3d %-14s %-10s %-10s %-8d%n",
            no++, c.name, c.season, c.soilType, c.growthDays);
    }
    System.out.println(" └──────────────────────────────────────────────────────────┘ ");
}
}
//==========================================================
//MODULE 7 — MarketDemandModule
//Sorts crops by demand (Bubble Sort), shows market reports
//==========================================================
class MarketDemandModule {
// Helper labels
static String getDemandLabel(int score) {
    if (score >= 80) return "● HIGH";
    if (score >= 60) return "□ MEDIUM";
    return "□ LOW";
}
static String getTrendLabel(int trend) {
    if (trend ==  1) return "▲ Rising";
    if (trend == -1) return "▼ Falling";
    return "── Stable";
```

```java
}
/**
 * Bubble Sort — sorts a List<Crop> by demandScore descending.
 * (Classic O(n²) sort — good for learning purposes)
 */
public static List<Crop> sortByDemand(HashMap<String, Crop> cropMap) {
    List<Crop> list = new ArrayList<>(cropMap.values());
    int n = list.size();
    // Bubble sort: repeatedly swap adjacent elements if out of order
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (list.get(j).demandScore < list.get(j + 1).demandScore) {
                Crop temp = list.get(j);
                list.set(j, list.get(j + 1));
                list.set(j + 1, temp);
            }
        }
    }
    return list;
}
/** Displays full ranked demand table */
public static void displayRankedDemand(HashMap<String, Crop> cropMap) {
    List<Crop> sorted = sortByDemand(cropMap);
    System.out.println("\n╔═══════════════════════════════════════════════════════════════════════════════╗");
    System.out.println("│        📈 CROP MARKET DEMAND RANKING (Sorted)        │");
    System.out.println("└═══════════════════════════════════════════════════════════════════════════════╝");
    System.out.printf("  %-5s %-14s %-8s %-12s %-10s %-10s%n",
        "Rank", "Crop", "Score", "Demand", "Price(₹)", "Trend");
    System.out.println("  ─────────────────────────────────────────────────────────────────────────────");
    for (int i = 0; i < sorted.size(); i++) {
        Crop c = sorted.get(i);
        System.out.printf("  %-5d %-14s %-8d %-12s %-10d %-10s%n",
            (i + 1), c.name, c.demandScore,
            getDemandLabel(c.demandScore),
            c.marketPrice,
            getTrendLabel(c.priceTrend));
    }
    System.out.println("  ─────────────────────────────────────────────────────────────────────────────");
}
/** Market summary: best/worst/average demand + export crops */
public static void displayMarketSummary(HashMap<String, Crop> cropMap) {
    List<Crop> list = new ArrayList<>(cropMap.values());
    Crop best  = list.get(0);
    Crop worst = list.get(0);
    int sum    = 0;
    for (Crop c : list) {
        if (c.demandScore > best.demandScore)  best  = c;
        if (c.demandScore < worst.demandScore) worst = c;
        sum += c.demandScore;
    }
    double avg = (double) sum / list.size();
    System.out.println("\n╔═══════════════════════════════════════════════════╗");
    System.out.println("│        📊 MARKET DEMAND SUMMARY        │");
    System.out.println("└═══════════════════════════════════════════════════╝");
    System.out.println("  🏆 Highest Demand : " + best.name
        + " (Score: " + best.demandScore + ")");
```

```java
    System.out.println("  📉 Lowest Demand  : " + worst.name
        + " (Score: " + worst.demandScore + ")");
    System.out.printf ("  📐 Average Demand : %.1f / 100%n", avg);
    System.out.println("\n  ── Crops With Export Potential ──");
    for (Crop c : list) {
        if (c.exportDemand) {
            System.out.println("    ✈ " + c.name
                + " | Price: ₹" + c.marketPrice + "/quintal");
        }
    }
    System.out.println("  ────────────────────────────────────────────────────────────");
}
/** Detailed market report for a single crop */
public static void displayCropMarketReport(Crop c) {
    System.out.println("\n╔═══════════════════════════════════════════════════════════════╗");
    System.out.println("║       📋 CROP MARKET REPORT              ║");
    System.out.println("╚═══════════════════════════════════════════════════════════════╝");
    System.out.println("  Crop Name      : " + c.name);
    System.out.println("  Demand Score   : " + c.demandScore + " / 100");
    System.out.println("  Demand Level   : " + getDemandLabel(c.demandScore));
    System.out.println("  Market Price   : ₹" + c.marketPrice + " / quintal");
    System.out.println("  Price Trend    : " + getTrendLabel(c.priceTrend));
    System.out.println("  Export Demand  : " + (c.exportDemand ? "✅ Yes" : "❌ No"));
    System.out.println("  ────────────────────────────────────────────────────────────");
    if (c.demandScore >= 80)
        System.out.println("  💡 Advice: HIGH demand — Good time to sell!");
    else if (c.demandScore >= 60)
        System.out.println("  💡 Advice: MEDIUM demand — Monitor market before selling.");
    else
        System.out.println("  💡 Advice: LOW demand — Consider holding stock for now.");
    System.out.println("  ────────────────────────────────────────────────────────────");
}
}
//==============================================================
//MAIN CLASS — Menu-driven entry point
//All logic lives in the modules above; Main just calls them.
//==============================================================
public class SmartFarmingSystem {
// ── Central crop database (HashMap — O(1) lookup by name) ──
static HashMap<String, Crop> cropMap = new HashMap<>();
/**
 * Initialise all crops.
 * Unified data from all four code modules.
 * Format: name, soil, season, climate,
 *       minTemp, maxTemp, idealTemp,
 *       waterNeed(0-100), minRainfall(mm), minMoisture(%), minHumidity(%),
 *       fertilityNeed(0-1), avgYield, growthDays, npkNeeds,
 *       marketDemand(0-1), demandScore, marketPrice, priceTrend, exportDemand
 */
static void initializeCrops() {
    cropMap.put("Rice",     new Crop("Rice",     "Clay",   "Kharif", "Hot & Humid",
                            20, 35, 28,   80, 1200, 70, 70,
                            0.7,  4.0, 150, "N:100 P:50 K:50",
                            0.9, 92,  2100,  1,  true));
    cropMap.put("Wheat",    new Crop("Wheat",    "Loamy",  "Rabi",   "Cool & Dry",
                            10, 25, 20,   50,  450, 40, 50,
                            0.6,  3.5, 120, "N:120 P:60 K:40",
                            0.8, 78,  2200,  1,  true));
    cropMap.put("Maize",    new Crop("Maize",    "Sandy",  "Kharif", "Warm",
```

```
                                18, 32, 25,   60,  600, 45, 55,
                                0.5,  5.0,  90, "N:150 P:60 K:40",
                                0.7, 65,  1900, -1, false));
    cropMap.put("Sugarcane", new Crop("Sugarcane", "Loamy",  "Kharif", "Hot & Wet",
                                20, 35, 28,   90, 2000, 80, 60,
                                0.8, 70.0, 365, "N:250 P:80 K:150",
                                0.6, 55,  3500,  0,  true));
    cropMap.put("Millet",    new Crop("Millet",    "Sandy",  "Kharif", "Semi-Arid",
                                25, 40, 32,   30,   30, 20, 30,
                                0.3,  2.0, 100, "N:60 P:30 K:20",
                                0.5, 50,  1800,  1, false));
    cropMap.put("Barley",    new Crop("Barley",    "Sandy",  "Rabi",   "Cool & Dry",
                                10, 25, 18,   40,   40, 30, 40,
                                0.4,  2.5, 110, "N:80 P:40 K:30",
                                0.5, 48,  1600,  0, false));
    cropMap.put("Soybean",   new Crop("Soybean",   "Loamy",  "Kharif", "Warm",
                                20, 30, 25,   55,  500, 50, 55,
                                0.5,  1.5, 100, "N:30 P:60 K:40",
                                0.6, 60,  4100,  0,  true));
    cropMap.put("Cotton",    new Crop("Cotton",    "Black",  "Kharif", "Warm & Dry",
                                20, 35, 28,   65,  700, 50, 55,
                                0.6,  1.8, 180, "N:100 P:50 K:50",
                                0.7, 70,  6200,  1,  true));
    cropMap.put("Tomato",    new Crop("Tomato",    "Red",    "All Season", "Warm",
                                18, 30, 24,   60,  600, 55, 60,
                                0.6, 25.0,  75, "N:120 P:60 K:60",
                                0.88, 88, 2000, -1, false));
    cropMap.put("Onion",     new Crop("Onion",     "Sandy",  "Rabi",   "Dry & Warm",
                                13, 28, 20,   40,  350, 35, 40,
                                0.5, 15.0, 120, "N:100 P:50 K:100",
                                0.95, 95, 2800,  1,  true));
}
// ── Utility: print a divider line ──
static void printDivider() {
    System.out.println("\n  ═══════════════════════════════════════════════════════════════");
}
// ── Main menu ──
public static void main(String[] args) {
    initializeCrops();
    Scanner sc = new Scanner(System.in);
    int choice;
    System.out.println("╔═══════════════════════════════════════════════════════════╗");
    System.out.println("║    🌱 SMART FARMING CROP RECOMMENDATION      ║");
    System.out.println("║          SYSTEM  v2.0                  ║");
    System.out.println("╚═══════════════════════════════════════════════════════════╝");
    do {
        // ── Print main menu ──
        System.out.println("\n  ┌─ MAIN MENU ──────────────────────────────┐");
        System.out.println("  │  1. Get Crop Recommendation          │");
        System.out.println("  │  2. View Crop Information            │");
        System.out.println("  │  3. Check Market Demand              │");
        System.out.println("  │  4. Pest Risk Alert                  │");
        System.out.println("  │  5. Fertility Advice                 │");
        System.out.println("  │  0. Exit                             │");
        System.out.println("  └──────────────────────────────────────────┘");
        System.out.print("\n  Enter choice: ");
        // Safe menu input
        try {
            choice = sc.nextInt();
            sc.nextLine();
```

```
        } catch (InputMismatchException e) {
            sc.next();
            choice = -1;
        }
        switch (choice) {
            // ────────────────────────────────────────────────────────────────────────────
            //  OPTION 1 — Crop Recommendation
            // ────────────────────────────────────────────────────────────────────────────
            case 1: {
                System.out.println("\n  ── Enter Your Farm Conditions ──");
                String soil     = ClimateInputHandler.getStringInput("  Soil Type
(Loamy/Clay/Sandy/Black/Red): ");
                double temp     = ClimateInputHandler.getDoubleInput("  Temperature (°C): ");
                double humidity = ClimateInputHandler.getDoubleInput("  Humidity (%): ");
                double water    = ClimateInputHandler.getDoubleInput("  Water Availability (0-100): ");
                double fertility= ClimateInputHandler.getDoubleInput("  Fertility Level (0.0 to 1.0): ");
                double rainfall = ClimateInputHandler.getDoubleInput("  Rainfall (mm): ");
                double waterAvail= ClimateInputHandler.getDoubleInput("  Available Water (mm, for
irrigation): ");
                if (!ClimateInputHandler.validateInputs(temp, humidity)) break;
                // Score and rank
                CropRecommendationEngine.calculateScores(cropMap, soil, temp, water, fertility);
                List<Crop> ranked = CropRecommendationEngine.getTopCrops(cropMap);
                CropRecommendationEngine.displayRecommendations(ranked);
                // Irrigation advice + drought warning
                WaterOptimizationModule.irrigationAdvice(waterAvail);
                WaterOptimizationModule.droughtWarning(rainfall, water);
                // If no crop scored well (score < 0.4), suggest water-efficient crops
                if (ranked.get(0).score < 0.4) {
                    System.out.println("\n  ⚠ No strong match found for given conditions.");
                    WaterOptimizationModule.suggestEfficientCrops(cropMap);
                }
                break;
            }
            // ────────────────────────────────────────────────────────────────────────────
            //  OPTION 2 — Crop Information (search / filter / list)
            // ────────────────────────────────────────────────────────────────────────────
            case 2: {
                System.out.println("\n  ── Crop Information ──");
                System.out.println("  a. Search by name");
                System.out.println("  b. Filter by season");
                System.out.println("  c. Filter by soil type");
                System.out.println("  d. View all crops");
                System.out.print("\n  Enter sub-choice (a/b/c/d): ");
                String sub = sc.nextLine().trim().toLowerCase();
                switch (sub) {
                    case "a": {
                        System.out.print("  Enter crop name: ");
                        String name = sc.nextLine();
                        Crop found = CropInformationModule.searchByName(cropMap, name);
                        if (found != null) CropInformationModule.displayCropProfile(found);
                        else System.out.println("  ⚠ Crop not found! Try: Wheat, Rice, Maize, etc.");
                        break;
                    }
                    case "b": {
                        System.out.print("  Enter season (Rabi/Kharif/All): ");
                        String season = sc.nextLine();
                        List<Crop> filtered = CropInformationModule.filterBySeason(cropMap, season);
                        CropInformationModule.displayCropList(filtered, "Season: " + season);
                        break;
```

```java
            }
            case "c": {
                System.out.print("  Enter soil type (Loamy/Sandy/Clay/Black/Red): ");
                String soil = sc.nextLine();
                List<Crop> filtered = CropInformationModule.filterBySoil(cropMap, soil);
                CropInformationModule.displayCropList(filtered, "Soil: " + soil);
                break;
            }
            case "d":
                CropInformationModule.displayAllCrops(cropMap);
                break;
            default:
                System.out.println("  Invalid sub-choice.");
        }
        break;
    }
    // ────────────────────────────────────────────────────────────────────────
    //  OPTION 3 — Market Demand
    // ────────────────────────────────────────────────────────────────────────
    case 3: {
        System.out.println("\n  ── Market Demand ──");
        System.out.println("  a. View all crops ranked by demand");
        System.out.println("  b. Market summary (best/worst/avg)");
        System.out.println("  c. Check report for a specific crop");
        System.out.print("\n  Enter sub-choice (a/b/c): ");
        String sub = sc.nextLine().trim().toLowerCase();
        switch (sub) {
            case "a":
                MarketDemandModule.displayRankedDemand(cropMap);
                break;
            case "b":
                MarketDemandModule.displayMarketSummary(cropMap);
                break;
            case "c": {
                System.out.print("  Enter crop name: ");
                String name = sc.nextLine();
                Crop found = CropInformationModule.searchByName(cropMap, name);
                if (found != null) MarketDemandModule.displayCropMarketReport(found);
                else System.out.println("  ⚠ Crop not found! Try: Wheat, Rice, Onion, etc.");
                break;
            }
            default:
                System.out.println("  Invalid sub-choice.");
        }
        break;
    }
    // ────────────────────────────────────────────────────────────────────────
    //  OPTION 4 — Pest Risk Alert
    // ────────────────────────────────────────────────────────────────────────
    case 4: {
        System.out.println("\n  ── Pest Risk Check ──");
        System.out.print("  Enter crop name: ");
        String cropName = sc.nextLine().trim();
        double temp     = ClimateInputHandler.getDoubleInput("  Temperature (°C): ");
        double humidity = ClimateInputHandler.getDoubleInput("  Humidity (%): ");
        if (!ClimateInputHandler.validateInputs(temp, humidity)) break;
        PestRiskModule.checkPestRisk(cropName, temp, humidity);
        break;
    }
    // ────────────────────────────────────────────────────────────────────────
```

```
            //  OPTION 5 — Fertility Advice
            // ────────────────────────────────────────────────────────────────────────────────
            case 5: {
                double fertility = ClimateInputHandler.getDoubleInput("  Enter Fertility Level (0.0 to 1.0): ");
                FertilityRecommendationModule.fertilityAlert(fertility);
                break;
            }
            case 0:
                System.out.println("\n  👋 Thank you for using Smart Farming System. Goodbye!");
                break;
            default:
                System.out.println("  ❌ Invalid choice. Please enter 0-5.");
        }
        printDivider();
    } while (choice != 0);
    sc.close();
}
}
```